

$$P(x) = \sum_{i=0}^n y_i \cdot l_i(x), \quad l_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

$$X_k = \sum_{j=0}^{n-1} a_j \cdot \omega^{jk} \pmod{p}$$

ω is an n -th root of unity in F_p

Security Review Report

NM-1063 - Safenet

September 29, 2026



NETHERMIND
SECURITY

Contents

1 Executive Summary	2
2 Audited Files	3
3 Summary of Issues	3
4 System Overview	4
5 Risk Rating Methodology	6
5.1 ZK Audit Issue Classification	6
6 Issues	7
6.1 [Low] keyGenCommit accepts the point at infinity as c[0]	7
6.2 [Info] Fee reward rounding leaves unclaimable dust in SentinelOracle	8
7 Documentation Evaluation	9
8 Test Suite Evaluation	10
8.1 Tests Output	10
9 About Nethermind	15

1 Executive Summary

This document presents the security review performed by [Nethermind Security](#) for the [Safenet](#) smart contracts. Safenet is a decentralized consensus and security framework designed to protect Gnosis Safe accounts through offchain threshold cryptography and onchain verification. The protocol leverages the FROST (Flexible Round-Optimized Schnorr Threshold) ciphersuite over secp256k1 (RFC 9591) to coordinate a validator set that reaches consensus on Safe transactions and system epochs. Protocol-gated transactions require valid multi-party cryptographic attestations or can proceed through a time-delayed escape hatch.

The audited code comprises 1673 lines of code written in the Solidity language. **The audit was performed using** (a) manual analysis of the codebase and (b) automated analysis tools. **Along this document, we report** two points of attention, where one is classified as Low, and one as Informational. **Fig. 1** summarizes the issues identified

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 concludes the document. Appendix describes the review of the deployment scripts.



Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (1), Undetermined (0), Informational (1), Best Practices (0). Distribution of status: Fixed (1), Acknowledged (1), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 18, 2026
Final Report	September 29, 2026
Response from Client	Regular responses during audit engagement
Repository	safenet
Audit Commit	898a5c5
Final Commit	0be9636
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	SentinelOracle.sol	291	108	37.1%	71	470
2	FROSTCoordinator.sol	302	373	123.5%	61	736
3	Consensus.sol	182	171	94.0%	45	398
4	Staking.sol	279	315	112.9%	112	706
5	libraries/Secp256k1.sol	116	157	135.3%	26	299
6	libraries/FROST.sol	201	223	110.9%	48	472
7	guard/SafenetGuard.sol	177	159	89.8%	33	369
8	interfaces/IFROSTCoordinatorCallback.sol	7	15	214.3%	3	25
9	interfaces/ISafenetGuard.sol	42	125	297.6%	24	191
10	interfaces/IOracle.sol	5	30	600.0%	4	39
11	interfaces/IConsensus.sol	71	133	187.3%	20	224
	Total	1673	1809	108.1%	447	3929

3 Summary of Issues

	Finding	Severity	Update
1	keyGenCommit accepts the point at infinity as <code>c[0]</code>	Low	Acknowledged
2	Fee reward rounding leaves unclaimable dust in SentinelOracle	Info	Fixed

4 System Overview

Safenet combines a transaction authorization layer, a threshold-signature protocol, an oracle-based policy mechanism, and a validator staking system. The main components considered in scope are SafenetGuard, SentinelOracle, FROSTCoordinator, FROST, Secp256k1, and Staking. Fig 2 depicts an overview of the main flows.

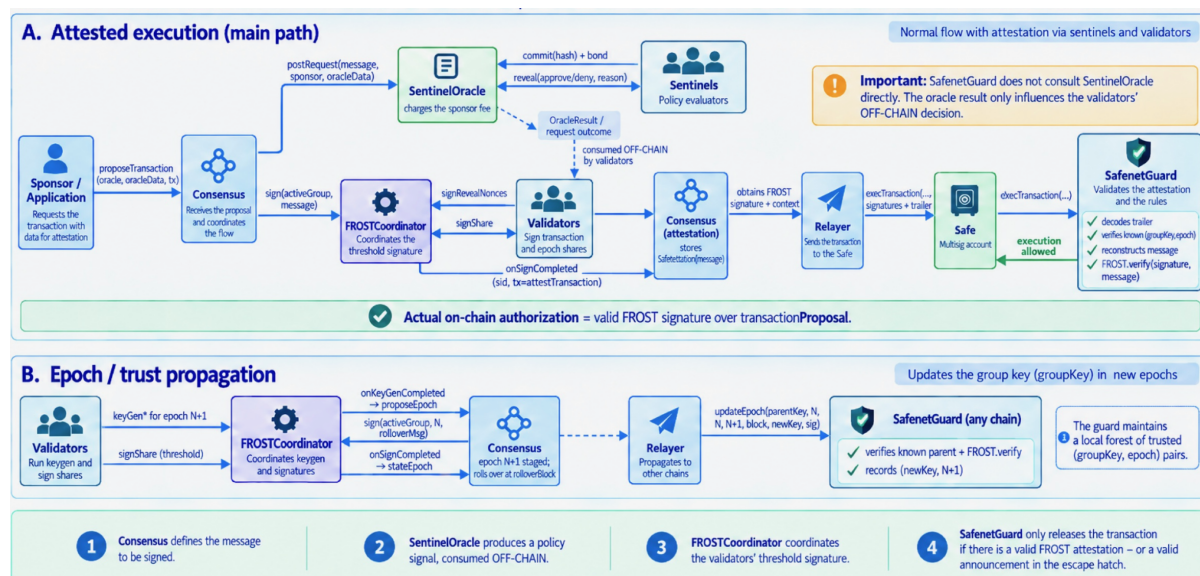


Fig. 2: System Overview of transaction proposal and Validators flow

SafenetGuard

SafenetGuard is the on-chain enforcement point for Safe transactions. Before execution, it validates that a transaction is authorized either by a valid Safenet attestation or through the time-windowed transaction announcement mechanism. For the attestation path, the contract decodes the attestation trailer, verifies that the provided (groupKey, epoch) pair is trusted, reconstructs the signed transaction proposal, and verifies the FROST signature. The contract also maintains the local set of trusted epoch group keys and supports FROST-authorized epoch rollovers.

SentinelOracle

SentinelOracle implements a commit-reveal voting mechanism in which registered sentinels evaluate requests and vote to approve or deny them. Requests define voting windows, fees, bonds, and potential slashing amounts. Conflicting votes move a request into a FROZEN state that may be resolved by the configured arbitrator, while resolved requests determine bond returns and fee rewards. The oracle does not directly authorize transactions in SafenetGuard; its result is consumed by the validator-side policy that determines whether validators participate in signing.

FROSTCoordinator

FROSTCoordinator is the on-chain coordination layer for FROST distributed key generation and threshold signing. It maintains FROST groups, participant information, group public keys, nonce commitments, signing sequences, and accumulated signature shares. The contract coordinates the DKG lifecycle, nonce preprocessing and revelation, and signature-share submission. A signing ceremony is considered complete only after the accumulated threshold signature is successfully verified. One design decision worth mentioning is that upon submitting key commitments during the DKG process, the proof of knowledge is not verified on-chain, and instead validators will listen to the commitment event and verify the proof of knowledge off-chain in the validator script.

FROST

The FROST library implements the cryptographic operations required by the threshold Schnorr signature scheme. It defines the FROST signature and signature-share structures and provides operations for computing challenges and binding factors, verifying individual signature shares, and verifying the final aggregate signature. In particular, `verifyShare()` validates a participant's signature contribution, while `verify()` validates the resulting threshold signature against the group public key.

Secp256k1

Secp256k1 provides the elliptic-curve primitives used by the FROST implementation and by contracts handling FROST public keys. It represents curve points, validates that points belong to the secp256k1 curve, performs point addition and serialization, and implements the relation checks required during FROST signature verification. The FROST library relies on these primitives when verifying both signature shares and complete signatures, while FROSTCoordinator uses them when constructing and maintaining participant and group public keys.

Staking

Staking provides the economic accounting layer associated with registered validators. Stakers may allocate SAFE_TOKEN to registered validators and later initiate time-delayed withdrawals. The contract tracks stake per validator and per staker, maintains withdrawal queues, and manages validator registration and configuration changes through delayed proposals. Its role is separate from the FROST signing logic: it manages validator-related stake and configuration rather than participating directly in signature generation or transaction authorization.

Component Interaction

At a high level, transaction proposals initiate both an oracle request and a FROST signing ceremony. SentinelOracle provides the policy outcome used by validators when deciding whether to sign, while FROSTCoordinator coordinates the threshold-signature ceremony. FROST performs the cryptographic verification using the lower-level Secp256k1 primitives. The resulting FROST attestation can subsequently be verified by SafenetGuard when the corresponding Safe transaction is executed. Staking operates alongside this flow as the validator stake-management layer.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Likelihood		
		Low	Medium	High
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract and circuit development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

5.1 ZK Audit Issue Classification

ZK protocols must adhere to the three principles of zero-knowledge: **Completeness**, **Soundness** and **Zero-Knowledge**. The table below describes the classification of issues in ZK circuits with their corresponding principles of zero-knowledge.

Property	Description	Classification
<i>Completeness</i>	A valid proof must always pass with an honest verifier	Over-constrained
<i>Soundness</i>	An invalid proof should pass only with negligible probability	Under-constrained
<i>Zero-Knowledge</i>	The proof must not reveal any information about the witness	Leakage

6 Issues

6.1 [Low] keyGenCommit accepts the point at infinity as c[0]

File(s): /src/FROSTCoordinator.sol, /src/libraries/Secp256k1.sol

Description: During FROST distributed key generation, each participant commits to the coefficients of a private polynomial. The first commitment, $c[0] = a_{i0} \cdot G$, represents that participant's contribution to the group secret. The group public key is derived by summing each participant's $c[0]$ commitment.

Elliptic-curve groups include an identity element, commonly called the point at infinity. Adding this point to another point has no effect. In the secp256k1 implementation used by the protocol, the point at infinity is represented internally as $(0, 0)$. Therefore, submitting $c[0] = (0, 0)$ is equivalent to contributing zero key material to the resulting group key.

`keyGenCommit()` accepts the point at infinity as $c[0]$, allowing a participant to register successfully while contributing no key material to the group public key. It verifies that the commitment vector has the expected length, but does not require $c[0]$ to be a non-identity curve point before adding it to `group.key`. This is depicted in the below snippet:

```

1  // from FROSTCoordinator.sol
2  function keyGenCommit(...){
3      // [...]
4      Secp256k1.requireNonZero(commitment.q);
5      require(commitment.c.length == state.threshold, InvalidGroupCommitment());
6      group.participants.register(msg.sender, poap);
7      group.state = state;
8      // @audit commitment c[0] added with no checks
9      group.key = Secp256k1.add(group.key, commitment.c[0]);
10     emit KeyGenCommitted(gid, msg.sender, commitment, committed);
11 }

```

The addition operation accepts $(0, 0)$ intentionally because the point at infinity is required as the identity element for elliptic-curve arithmetic. Consequently, relying on `Secp256k1.add()` as implicit validation is insufficient: invalid off-curve points are rejected, while $(0, 0)$ remains valid input to the addition routine. Below is the `_unpack()` function that checks the validity of the input to `Secp256k1.add()` and returns the points on the curve. Even though the point is not on the curve, $(0,0)$ must be accepted as it acts as the point of infinity thus passing the `require()` statement below.

```

1  // from Secp256k1.sol
2  function _unpack(Point memory p) private pure returns (uint256 x, uint256 y) {
3      x = p.x;
4      y = p.y;
5      bool valid = _satisfiesCurveEquation(x, y);
6      assembly ("memory-safe") {
7          // @audit iszero will return true(1) for (0,0) passing the requirement below
8          valid := or(iszero(or(x, y)), valid)
9      }
10     require(valid, NotOnCurve());
11 }

```

A participant can therefore submit $c[0] = (0, 0)$ and complete the commitment phase without contributing to the group secret. This reduces the effective number of key contributors while leaving the total number of registered participants unchanged. For example, a group configured with five participants may contain only four participants that actually contribute key material, weakening the intended threshold security assumptions without changing the group membership or configured threshold.

Recommendation: Explicitly reject the identity point for $c[0]$ in `keyGenCommit()` before updating the group key.

Status: Acknowledged

Update from Client: The off-chain proof of knowledge will catch this edge case.

6.2 [Info] Fee reward rounding leaves unclaimable dust in SentinelOracle

File(s): [/src/SentinelOracle.sol](#)

Description: When a request is resolved, the distributable fee is shared equally among the sentinels that voted for the winning outcome. The reward for each winning sentinel is calculated using integer division:

```
1  function claim(bytes32 requestId) external {
2      //..
3      uint96 feeReward = request.calcFeeReward(state, vote); // @audit calculates integer division
4      uint96 bondReturn = commitment.bondAmount - request.slashAmountFor(state, vote);
5      uint256 totalClaim = uint256(bondReturn) + feeReward;
6      if (totalClaim > 0) {
7          FEE_TOKEN.safeTransfer(msg.sender, totalClaim);
8      }
9
10     function calcFeeReward(...) {
11         //..
12         return self.progress.fee / winningSideCount;
13     }
```

If `self.progress.fee` is not evenly divisible by `winningSideCount`, the remainder is not allocated to any sentinel.

The same behavior can also occur when a disputed request is resolved through `resolveDispute()` whenever the distributable fee is not evenly divisible by the number of winning sentinels.

Over time, these rounding remainders can accumulate as unclaimable dust in the contract.

Recommendation(s): Consider explicitly accounting for rounding remainders when distributing fee rewards, for example by assigning the remainder to the `protocolFundsReceiver` or deterministically to one of the winning sentinels.

Status: Fixed

Update from Client: Fixed in commit [2f31511](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information describing software's functionality, architecture, design, and implementation. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Protocols can use various types of software documentation for their circuits. Some of the most common types include:

Circuit Specifications: A documentation that outlines the purpose, functionality, and constraints of the circuit:

- i) High-level description - What the circuit is designed to achieve, e.g., proving membership in a Merkle tree, verifying a signature, etc. Description about inputs and outputs (public and private signals);
- ii) Mathematical model - Description of the mathematical computations and proofs being implemented. In addition, describe the cryptographic primitives that are used, e.g., Poseidon hash, EdDSA verification.
- iii) Expected behavior - Present a detailed explanation of how the circuit behaves under normal, edge, and adversarial conditions.

Documentation of the Input and Output: The documentation should describe details of the inputs and outputs of the circuit.

- i) Public inputs: List of public inputs and their expected values and ranges. Also, clarify constraints on the public inputs (e.g., nonzero).
- ii) Private inputs: List of private input signals and their roles in the circuit. The documentation should also include validation requirements for private inputs within the circuit.
- iii) Outputs: A description of the outputs, their intended use and their relationship with the inputs.

Performance Metrics: Describe details on the prover and verifier performance such as, time taken to generate proofs, memory usage during proof generation, time taken to verify proofs, etc.

Testing Documentation: Testing documentation is a document that provides information about how the circuits were tested. It includes details on the test cases that were used, coverage metrics showing which parts of the circuit have been tested, and any issues that were identified during testing.

Technical whitepaper: A technical whitepaper is a comprehensive document that describes the design and technical details of the protocol. It includes information on the purpose of the protocol, contracts, its architecture, its components, and how they interact with each other;

Summary: These types of documentation are essential for circuit development and maintenance. They help ensure that the circuit is properly designed, implemented, and tested. The documentation provides a reference for developers who need to modify the circuit in the future.

Remarks about the Safenet documentation

The Safenet team provided audit-specific documentation, detailing the core contracts and files, outlining the flow step by step with comments in the code. Furthermore, the Safenet team thoroughly addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
> forge test

Ran 1 test for test/libraries/SafeTransaction.t.sol:SafeTransactionTest
[PASS] test_SafeTransactionHash() (gas: 4743)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.71ms (850.33µs CPU time)

Ran 3 tests for test/libraries/SafeId.t.sol:SafeIdTest
[PASS] test_Create_ConcatenatesChainIdAndSafe() (gas: 434)
[PASS] test_Create_MaxChainId() (gas: 419)
[PASS] test_Create_RevertsWhenChainIdOverflows() (gas: 4967)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 6.80ms (1.96ms CPU time)

Ran 8 tests for test/libraries/SignatureExtension.t.sol:SignatureExtensionTest
[PASS] test_has_falseForDifferentTypeHash() (gas: 4106)
[PASS] test_has_falseWhenShorterThanWord() (gas: 3930)
[PASS] test_has_trueForMatchingTypeHash() (gas: 4190)
[PASS] test_payload_ignoresOwnerSignatureLength() (gas: 16725)
[PASS] test_payload_revertsOnTypeHashMismatch() (gas: 9089)
[PASS] test_payload_revertsWhenLengthOverrunsFront() (gas: 9227)
[PASS] test_payload_revertsWhenTooShort() (gas: 7384)
[PASS] test_payload_roundTripsAcrossSizes() (gas: 22882)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 486.38µs (381.71µs CPU time)

Ran 3 tests for test/AlwaysApproveOracle.t.sol:AlwaysApproveOracleTest
[PASS] test_PostRequest_EmitsApprovedTrue() (gas: 19452)
[PASS] test_PostRequest_EmitsOracleResult_Immediately() (gas: 17402)
[PASS] test_PostRequest_SponsorIsPassedAddress() (gas: 18986)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.74ms (917.13µs CPU time)

Ran 12 tests for test/Consensus.t.sol:ConsensusTest
[PASS] test_AttestTransaction_DoubleAttest_Reverts() (gas: 119892)
[PASS] test_AttestTransaction_StoresAndEmits() (gas: 125170)
[PASS] test_GetCurrentEpochs_GenesisInfo() (gas: 9330)
[PASS] test_GetCurrentEpochs_MultipleEpochs() (gas: 146056)
[PASS] test_GetCurrentEpochs_NewEpoch() (gas: 75083)
[PASS] test_GetCurrentEpochs_StagedEpoch() (gas: 71887)
[PASS] test_GetEpochGroup_EmptyForUnknownEpoch() (gas: 8045)
[PASS] test_GetEpochGroup_ExistingGroup() (gas: 11997)
[PASS] test_GetTransactionAttestationByHash() (gas: 118690)
[PASS] test_ProposeTransaction_AlreadyAttested_Reverts() (gas: 125794)
[PASS] test_ProposeTransaction_EmitsEvent() (gas: 114086)
[PASS] test_updateValidatorStaker() (gas: 38563)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 7.79ms (765.50µs CPU time)

Ran 10 tests for test/SimpleOracle.t.sol:SimpleOracleTest
[PASS] test_Approve_ClearsRequest() (gas: 30875)
[PASS] test_Approve_EmitsOracleResult_True() (gas: 31732)
[PASS] test_Approve_NotApprover_Reverts() (gas: 36024)
[PASS] test_Approve_RequestNotPending_Reverts() (gas: 13828)
[PASS] test_PostRequest_AlreadyPending_Reverts() (gas: 37671)
[PASS] test_PostRequest_RecordsSponsor() (gas: 28857)
[PASS] test_Reject_ClearsRequest() (gas: 30963)
[PASS] test_Reject_EmitsOracleResult_False() (gas: 31765)
[PASS] test_Reject_NotApprover_Reverts() (gas: 36024)
[PASS] test_Reject_RequestNotPending_Reverts() (gas: 13851)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 450.88µs (196.46µs CPU time)

Ran 6 tests for test/libraries/ConsensusMessages.t.sol:ConsensusMessagesTest
[PASS] test_EpochRollover() (gas: 784)
[PASS] test_TransactionProposal() (gas: 720)
[PASS] test_TransactionProposalTypehash() (gas: 244)
[PASS] test_TransactionProposal_SentinelParityVector() (gas: 1663)
[PASS] test_messageFamiliesDoNotCollide() (gas: 1279)
[PASS] test_transactionProposal_injectiveInEachField() (gas: 3461)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 123.46µs (51.42µs CPU time)

Ran 2 tests for test/libraries/FROSTNonceCommitmentSet.t.sol:FROSTNonceCommitmentSetTest
```

```
[PASS] test_CannotVerifyPastSequences() (gas: 1279561)
[PASS] test_ConsecutiveCommit() (gas: 68729)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 8.35ms (3.51ms CPU time)

Ran 7 tests for test/libraries/FROST.t.sol:FROSTTest
[PASS] test_BindingFactors() (gas: 48622)
[PASS] test_Challenge() (gas: 3628)
[PASS] test_Identifier() (gas: 2299)
[PASS] test_KeyGenChallenge() (gas: 5598)
[PASS] test_Nonce() (gas: 2276)
[PASS] test_Verify() (gas: 7648)
[PASS] test_Verify_revertsOnNonCanonicalScalar() (gas: 7078)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 10.38ms (5.53ms CPU time)

Ran 11 tests for test/libraries/Secp256k1.t.sol:Secp256k1Test
[PASS] test_Add() (gas: 16613)
[PASS] test_AddDouble() (gas: 13756)
[PASS] test_AddNegative() (gas: 9129)
[PASS] test_AddZero() (gas: 8585)
[PASS] test_AddZeroDoesNotAlias() (gas: 8283)
[PASS] test_InvalidMulMulAddWitness() (gas: 13412)
[PASS] test_MulMulAdd() (gas: 19021)
[PASS] test_MulMulAddIdentity() (gas: 9669)
[PASS] test_MulMulAddIdentityPoint() (gas: 14159)
[PASS] test_NotOnCurve() (gas: 24853)
[PASS] test_UnsupportedMulMulAddInput() (gas: 38648)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.76ms (2.64ms CPU time)

Ran 9 tests for test/libraries/TransactionAnnouncement.t.sol:TransactionAnnouncementTest
[PASS] test_announce_renewsExpired() (gas: 29539)
[PASS] test_announce_revertsOnWindowOverflow() (gas: 8047)
[PASS] test_announce_revertsWhilePendingOrActive() (gas: 29313)
[PASS] test_announce_setsWindow() (gas: 25352)
[PASS] test_cancel_deletes() (gas: 16424)
[PASS] test_cancel_revertsIfMissing() (gas: 6936)
[PASS] test_consume_falseAfterExpiry() (gas: 29700)
[PASS] test_consume_onlyWithinWindow() (gas: 21294)
[PASS] test_hash_pinsFieldLayout() (gas: 3739)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 215.92µs (129.46µs CPU time)

Ran 5 tests for test/Validator7702Account.t.sol:Validator7702AccountTest
[PASS] test_ExecuteAppliesPerCallGasLimit() (gas: 42709)
[PASS] test_ExecuteContinuesPastFailure_EmitsCallFailed() (gas: 104362)
[PASS] test_Execute_ForwardsToPerCallTargets() (gas: 150967)
[PASS] test_Execute_NotSelf_Reverts() (gas: 18319)
[PASS] test_Receive_AcceptsNativeToken() (gas: 20925)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.81ms (589.33µs CPU time)

Ran 66 tests for test/SentinelOracle.t.sol:SentinelOracleTest
[PASS] test_AddSentinel_OnlyGovernance() (gas: 41545)
[PASS] test_BondConfig_RescheduleBeforeMaturity_OverwritesPending() (gas: 33615)
[PASS] test_BondConfig_ScheduleApplyRoundTrip() (gas: 43625)
[PASS] test_CharterEns_RoundTrips() (gas: 13776)
[PASS] test_Conflict_ArbitrationDaoFeeShare_SponsorRefundUnaffected() (gas: 349619)
[PASS] test_Conflict_MidRangeSlashingMultiplier_LosingSentinelReclaimsPartialRemainder() (gas: 355011)
[PASS] test_Conflict_MinimumSlashingMultiplier_LosingSentinelReclaimsRemainder() (gas: 325177)
[PASS] test_Conflict_SetsStateFrozen() (gas: 344019)
[PASS] test_Conflict_WithUnrevealedCommitter_NoFundsTrappedOrDoubleSlashed() (gas: 436864)
[PASS] test_DaoFeeShare_RescheduleBeforeMaturity_OverwritesPending() (gas: 47825)
[PASS] test_DaoFeeShare_ScheduleApplyRoundTrip() (gas: 54327)
[PASS] test_DoubleCommit_Reverts() (gas: 202476)
[PASS] test_DoubleReveal_Reverts() (gas: 207627)
[PASS] test_Fee_RescheduleBeforeMaturity_OverwritesPending() (gas: 29488)
[PASS] test_Fee_ScheduleApplyRoundTrip() (gas: 37088)
[PASS] test_Finalize_EagerlyAppliesPendingProtocolFundsReceiver() (gas: 401187)
[PASS] test_Finalize_UnrevealedBondAndDaoCutDoNotOverflowWithMaxUint96Amounts() (gas: 3948951)
[PASS] test_HashCommitment_ParityWithRustImplementation() (gas: 7885)
[PASS] test_MarkOutOfScope_EmitsEvent() (gas: 301907)
[PASS] test_MarkOutOfScope_NoDeadlineCheck() (gas: 305670)
[PASS] test_MarkOutOfScope_OnlyArbitrator() (gas: 302275)
[PASS] test_MarkOutOfScope_RefundsFeeAndBondsInFull() (gas: 313322)
[PASS] test_MarkOutOfScope_RevertsWhenNotFrozen() (gas: 133370)
[PASS] test_NoCommitments_FeeRefunded() (gas: 134090)
[PASS] test_NoReveals_BondsAndFeeRefundedInFull() (gas: 290139)
[PASS] test_NoReveals_DoubleClaimReverts() (gas: 193381)
```

```
[PASS] test_PartialReveal_PartialSlashingMultiplier_NonRevealerReclaimsRemainder() (gas: 408373)
[PASS] test_PartialReveal_ResolvesAndSlashesNonRevealer() (gas: 408414)
[PASS] test_PostRequest_EagerlyAppliesPendingBondMultiplier() (gas: 153532)
[PASS] test_PostRequest_EagerlyAppliesPendingDaoFeeShare() (gas: 167037)
[PASS] test_PostRequest_EagerlyAppliesPendingFee() (gas: 148185)
[PASS] test_ProtocolFundsReceiver_RescheduleBeforeMaturity_OverwritesPending() (gas: 40579)
[PASS] test_ProtocolFundsReceiver_ScheduleApplyRoundTrip() (gas: 42952)
[PASS] test_RemoveSentinel_OnlyGovernance() (gas: 24032)
[PASS] test_ResolveDispute_EmptyContext_Accepted() (gas: 321621)
[PASS] test_ResolveDispute_SlashedDoesNotOverflowWithMaxUint96Amounts() (gas: 3946951)
[PASS] test_Reveal_AfterRevealDeadline_Reverts() (gas: 204104)
[PASS] test_Reveal_BeforeCommitDeadline_Reverts() (gas: 204418)
[PASS] test_Reveal_EmptyReason_Accepted() (gas: 203635)
[PASS] test_Reveal_ReasonEmittedVerbatim() (gas: 208821)
[PASS] test_Reveal_WithoutCommit_Reverts() (gas: 139573)
[PASS] test_Reveal_WrongReason_Reverts() (gas: 199965)
[PASS] test_Reveal_WrongSalt_Reverts() (gas: 201429)
[PASS] test_Reveal_WrongVote_Reverts() (gas: 199738)
[PASS] test_ScheduleBondConfig_OnlyGovernance() (gas: 24881)
[PASS] test_ScheduleBondConfig_RequestsInFlightKeepSnapshottedSlashAmount() (gas: 145570)
[PASS] test_ScheduleBondConfig_SlashingMultiplierAboveBondMultiplier_Reverts() (gas: 12633)
[PASS] test_ScheduleBondConfig_SlashingMultiplierBelowOne_Reverts() (gas: 11476)
[PASS] test_ScheduleBondConfig_ZeroBondMultiplier_Reverts() (gas: 12373)
[PASS] test_ScheduleDaoFeeShare_AboveDenominator_Reverts() (gas: 12331)
[PASS] test_ScheduleDaoFeeShare_OnlyGovernance() (gas: 41847)
[PASS] test_ScheduleDaoFeeShare_RequestsInFlightKeepSnapshottedShare() (gas: 162823)
[PASS] test_ScheduleFee_OnlyGovernance() (gas: 23771)
[PASS] test_ScheduleFee_RequestsInFlightKeepSnapshottedFee() (gas: 145989)
[PASS] test_ScheduleFee_Zero_Reverts() (gas: 11341)
[PASS] test_ScheduleProtocolFundsReceiver_OnlyGovernance() (gas: 49702)
[PASS] test_ScheduleProtocolFundsReceiver_ZeroAddress_Reverts() (gas: 11883)
[PASS] test_TimeoutArbitration_EmitsEvent() (gas: 298085)
[PASS] test_TimeoutArbitration_Permissionless() (gas: 302614)
[PASS] test_TimeoutArbitration_RefundsFeeAndBondsInFull() (gas: 309465)
[PASS] test_TimeoutArbitration_RevertsBeforeDeadline() (gas: 296874)
[PASS] test_TimeoutArbitration_RevertsWhenNotFrozen() (gas: 131122)
[PASS] test_TimeoutArbitration_UnrevealedCommitter_NoDoubleRefund() (gas: 400177)
[PASS] test_UnanimousApprove_DaoFeeShareCutGoesToProtocolFundsReceiver() (gas: 350187)
[PASS] test_UnanimousApprove_FeeDistributedAndBondsReturned() (gas: 314888)
[PASS] test_UnanimousDeny_FeeDistributedToDenySentinels() (gas: 215470)
Suite result: ok. 66 passed; 0 failed; 0 skipped; finished in 13.27ms (17.57ms CPU time)
```

Ran 2 tests for test/FROSTCoordinator.t.sol:FROSTCoordinatorTest

```
[PASS] test_KeyGen() (gas: 1751643)
[PASS] test_Sign() (gas: 1978556)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 26.79ms (26.81ms CPU time)
```

Ran 69 tests for test/Staking.t.sol:StakingTest

```
[PASS] test_BatchValidatorChanges() (gas: 104227)
[PASS] test_ClaimWithdrawal() (gas: 216588)
[PASS] test_ClaimWithdrawal_ExactlyAtClaimableTime() (gas: 207188)
[PASS] test_ClaimWithdrawal_Multiple() (gas: 262640)
[PASS] test_Constructor() (gas: 18331)
[PASS] test_Constructor_Event() (gas: 3386882)
[PASS] test_DeregisterUnregisteredValidator() (gas: 45765)
[PASS] test_DeregisterValidator() (gas: 48387)
[PASS] test_DeregisteredValidator_CannotStakeNew() (gas: 49084)
[PASS] test_DeregisteredValidator_ExistingStakesCanWithdraw() (gas: 252608)
[PASS] test_ExecuteValidatorChanges() (gas: 66134)
[PASS] test_ExecuteWithdrawDelayChange() (gas: 37676)
[PASS] test_GetNextClaimableWithdrawal() (gas: 253777)
[PASS] test_GetPendingWithdrawals() (gas: 311491)
[PASS] test_GetPendingWithdrawals_EmptyQueue() (gas: 11910)
[PASS] test_GetPendingWithdrawals_ThreeOrMoreWithdrawals() (gas: 429989)
[PASS] test_InitiateWithdrawalAtPosition_EmptyQueue() (gas: 250223)
[PASS] test_InitiateWithdrawalAtPosition_Head() (gas: 329589)
[PASS] test_InitiateWithdrawalAtPosition_Middle() (gas: 400047)
[PASS] test_InitiateWithdrawalAtPosition_Tail() (gas: 309300)
[PASS] test_InitiateWithdrawal_FullStake() (gas: 200266)
[PASS] test_InitiateWithdrawal_Head() (gas: 259904)
[PASS] test_InitiateWithdrawal_ReducedDelay_InsertsAtHead() (gas: 332512)
[PASS] test_InitiateWithdrawal_ReducedDelay_InsertsInMiddle() (gas: 393925)
[PASS] test_InitiateWithdrawal_Tail() (gas: 312977)
[PASS] test_ProposeValidators() (gas: 56917)
[PASS] test_ProposeValidators_OverwritesPending() (gas: 60936)
```

```
[PASS] test_ProposeWithdrawDelay() (gas: 46140)
[PASS] test_RecoverTokens() (gas: 906463)
[PASS] test_RecoverTokens_ExcessSafeToken() (gas: 68801)
[PASS] test_RecoverTokens_WithStakedAndPending() (gas: 300684)
[PASS] test_RegisterAlreadyRegisteredValidator() (gas: 45712)
[PASS] test_RevertWhen_ClaimWithdrawal_QueueEmpty() (gas: 147952)
[PASS] test_RevertWhen_Constructor_InvalidDelay() (gas: 70601)
[PASS] test_RevertWhen_Constructor_ZeroConfigTimeDelay() (gas: 70487)
[PASS] test_RevertWhen_Constructor_ZeroSafeToken() (gas: 68175)
[PASS] test_RevertWhen_Constructor_ZeroWithdrawDelay() (gas: 70488)
[PASS] test_RevertWhen_ExecuteValidatorChanges_InvalidParams() (gas: 63935)
[PASS] test_RevertWhen_ExecuteValidatorChanges_NoProposal() (gas: 15310)
[PASS] test_RevertWhen_ExecuteValidatorChanges_Timelock() (gas: 53077)
[PASS] test_RevertWhen_ExecuteWithdrawDelayChange_NoProposal() (gas: 11069)
[PASS] test_RevertWhen_ExecuteWithdrawDelayChange_Timelock() (gas: 42998)
[PASS] test_RevertWhen_InitiateWithdrawalAtPosition_InvalidOrdering() (gas: 369080)
[PASS] test_RevertWhen_InitiateWithdrawalAtPosition_InvalidPrevious() (gas: 182437)
[PASS] test_RevertWhen_InitiateWithdrawal_InsufficientStake() (gas: 147113)
[PASS] test_RevertWhen_InitiateWithdrawal_ZeroAmount() (gas: 146835)
[PASS] test_RevertWhen_ProposeValidators_EmptyArray() (gas: 15193)
[PASS] test_RevertWhen_ProposeValidators_LengthMismatch() (gas: 17819)
[PASS] test_RevertWhen_ProposeValidators_NotOwner() (gas: 18782)
[PASS] test_RevertWhen_ProposeValidators_Self() (gas: 19178)
[PASS] test_RevertWhen_ProposeValidators_ZeroAddress() (gas: 18693)
[PASS] test_RevertWhen_ProposeWithdrawDelay_Invalid() (gas: 14532)
[PASS] test_RevertWhen_ProposeWithdrawDelay_NotOwner() (gas: 14878)
[PASS] test_RevertWhen_RecoverTokens_InvalidAddress() (gas: 16897)
[PASS] test_RevertWhen_RecoverTokens_NotOwner() (gas: 892041)
[PASS] test_RevertWhen_RecoverTokens_StakedFunds() (gas: 158308)
[PASS] test_RevertWhen_Stake_InsufficientAllowance() (gas: 121724)
[PASS] test_RevertWhen_Stake_InsufficientBalance() (gas: 172323)
[PASS] test_RevertWhen_Stake_InvalidValidatorAddress() (gas: 12111)
[PASS] test_RevertWhen_Stake_UnregisteredValidator() (gas: 16576)
[PASS] test_RevertWhen_Stake_ZeroAmount() (gas: 14255)
[PASS] test_Stake() (gas: 158308)
[PASS] test_Stake_Incremental() (gas: 169913)
[PASS] test_Stake_MultipleStakers() (gas: 210797)
[PASS] test_Stake_MultipleValidators() (gas: 258207)
[PASS] test_Stake_TotalAmount() (gas: 211229)
[PASS] test_StateConsistency_AfterFullWithdrawalAndClaim() (gas: 218428)
[PASS] test_StateConsistency_MultipleStakersMultipleValidators() (gas: 520141)
[PASS] test_StateConsistency_TokenBalanceMatchesAccountedFunds() (gas: 318396)
Suite result: ok. 69 passed; 0 failed; 0 skipped; finished in 19.55ms (7.77ms CPU time)
```

```
Ran 13 tests for test/libraries/EpochRollover.t.sol:EpochRolloverTest
[PASS] test_initialize_revertsOnZeroKey() (gas: 5660)
[PASS] test_initialize_seedsAndEmits() (gas: 32970)
[PASS] test_isKnown_exactOnPair() (gas: 42025)
[PASS] test_rollover_branchingSameEpoch() (gas: 131993)
[PASS] test_rollover_chainDepthTwo() (gas: 132036)
[PASS] test_rollover_idempotentResubmission() (gas: 90327)
[PASS] test_rollover_recordsChildAndEmits() (gas: 83704)
[PASS] test_rollover_revertsKnownKeyWrongEpoch() (gas: 48041)
[PASS] test_rollover_revertsOnMismatchedMessageField() (gas: 59610)
[PASS] test_rollover_revertsOnZeroNewKey() (gas: 43877)
[PASS] test_rollover_revertsUnknownParent() (gas: 47800)
[PASS] test_rollover_revertsWhenNotAdvancing() (gas: 53052)
[PASS] test_rollover_sameKeyTwoEpochs() (gas: 132058)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 27.68ms (11.78ms CPU time)
```

```
Ran 71 tests for test/SafenetGuard.t.sol:SafenetGuardTest
[PASS] test_announceTransaction_emitsEvent() (gas: 82797)
[PASS] test_announceTransaction_revertsOnDuplicate() (gas: 95330)
[PASS] test_announcementHash_separatesSameLengthData() (gas: 12437)
[PASS] test_announcement_activeCannotBeOverwritten() (gas: 96059)
[PASS] test_announcement_callCannotAuthorizeDelegatecall() (gas: 92602)
[PASS] test_announcement_cancelIsImmediateAndBlocksExecution() (gas: 95368)
[PASS] test_announcement_cannotBeConsumedByAnotherSafe() (gas: 363800)
[PASS] test_announcement_consumedEvenWhenInnerCallFails() (gas: 144137)
[PASS] test_announcement_consumedOnUse() (gas: 100126)
[PASS] test_announcement_emitsExecutedViaAllowance() (gas: 84582)
[PASS] test_announcement_executableAtWindowEnd() (gas: 77824)
[PASS] test_announcement_expiredIsRenewableInPlace() (gas: 105491)
[PASS] test_announcement_notConsumedByAttestedExecution() (gas: 121410)
[PASS] test_announcement_passesAfterDelay() (gas: 78058)
```

```
[PASS] test_announcement_revertsAfterWindowExpires() (gas: 97508)
[PASS] test_announcement_revertsBeforeDelay() (gas: 90989)
[PASS] test_announcement_singleUseUnderReentrancy() (gas: 753223)
[PASS] test_announcement_survivesNonceAdvanceAndExecutesNonceFree() (gas: 157130)
[PASS] test_announcement_zeroGasInnerRevertRollsBackConsumption() (gas: 162162)
[PASS] test_cancelAnnouncement_externalCallerCannotCancelSafesAnnouncement() (gas: 86765)
[PASS] test_cancelAnnouncement_revertsIfNotPending() (gas: 50627)
[PASS] test_cancelAnnouncement_worksAfterExpiry() (gas: 83991)
[PASS] test_checkTransaction_approvedHashPassesWithAttestation() (gas: 67619)
[PASS] test_checkTransaction_attestedWithNonDefaultGasFields() (gas: 70787)
[PASS] test_checkTransaction_autoAllowExcludesGasPricedRefund() (gas: 114157)
[PASS] test_checkTransaction_autoAllowsAnnounceTransactionCall() (gas: 81825)
[PASS] test_checkTransaction_autoAllowsCancelAnnouncementCall() (gas: 79815)
[PASS] test_checkTransaction_contractSignaturePassesWithAttestation() (gas: 416528)
[PASS] test_checkTransaction_doesNotAutoAllowDelegatecall() (gas: 49717)
[PASS] test_checkTransaction_doesNotAutoAllowNonZeroValue() (gas: 49390)
[PASS] test_checkTransaction_doesNotAutoAllowOtherSelectors() (gas: 48114)
[PASS] test_checkTransaction_doesNotAutoAllowShortData() (gas: 49238)
[PASS] test_checkTransaction_mixedOwnerEncodingsWithTrailer() (gas: 539114)
[PASS] test_checkTransaction_passesWithInlineAttestation() (gas: 70114)
[PASS] test_checkTransaction_passesWithOracleData() (gas: 70847)
[PASS] test_checkTransaction_replayProtectedByNonce() (gas: 95211)
[PASS] test_checkTransaction_revertsWhenGenesisKeyClaimedAtWrongEpoch() (gas: 55828)
[PASS] test_checkTransaction_revertsWhenNoAttestation() (gas: 44771)
[PASS] test_checkTransaction_revertsWhenOracleDataTampered() (gas: 64303)
[PASS] test_checkTransaction_revertsWithUntrustedKey() (gas: 56180)
[PASS] test_checkTransaction_twoOfThreePassesWithAttestation() (gas: 374172)
[PASS] test_constructor_acceptsMaxUint64Timing() (gas: 1557015)
[PASS] test_constructor_derivesConsensusDomainSeparator() (gas: 7748)
[PASS] test_constructor_emitsEpochInitialized() (gas: 1560481)
[PASS] test_constructor_revertsOnInvalidGroupKey() (gas: 39891)
[PASS] test_constructor_revertsOnOversizedDelay() (gas: 40834)
[PASS] test_constructor_revertsOnOversizedWindow() (gas: 39531)
[PASS] test_constructor_revertsOnZeroAllowTxDelay() (gas: 39828)
[PASS] test_constructor_revertsOnZeroAllowTxWindow() (gas: 41539)
[PASS] test_constructor_revertsOnZeroConsensusAddress() (gas: 39708)
[PASS] test_constructor_seedsGenesisPairInForest() (gas: 20019)
[PASS] test_forest_historicalKeySignsFutureTransaction() (gas: 153707)
[PASS] test_forest_historicalParentCreatesSiblingBranch() (gas: 155006)
[PASS] test_getAnnouncementWindow_returnsBothBounds() (gas: 80197)
[PASS] test_integration_inlineAttestation_revertsWithTamperedSignature() (gas: 64382)
[PASS] test_integration_updateEpoch_revertsWithTamperedSignature() (gas: 27720)
[PASS] test_safeTransactionHash_matchesDeployedSafe() (gas: 18428)
[PASS] test_supportsInterface() (gas: 9676)
[PASS] test_trailer_malformedTruncatedReverts() (gas: 44158)
[PASS] test_trailer_noMagicTreatedAsAbsent() (gas: 46304)
[PASS] test_trailer_signatureEndingInPayloadLengthTreatedAsAbsent() (gas: 46348)
[PASS] test_trailer_wrongPayloadSizeReverts() (gas: 43685)
[PASS] test_updateEpoch_emitsRolledOver() (gas: 54829)
[PASS] test_updateEpoch_isPermissionless() (gas: 56147)
[PASS] test_updateEpoch_keepsHistoricKeysValid() (gas: 112932)
[PASS] test_updateEpoch_recordsNewPairAndAcceptsItsAttestation() (gas: 113799)
[PASS] test_updateEpoch_revertsOnInvalidNewKey() (gas: 16846)
[PASS] test_updateEpoch_revertsUnknownParent() (gas: 20122)
[PASS] test_updateEpoch_revertsWhenNotAdvancing() (gas: 19860)
[PASS] test_updateEpoch_skippedEpochNeverBecomesTrusted() (gas: 100548)
[PASS] test_updateEpoch_supportsForkedBranches() (gas: 194333)
Suite result: ok. 71 passed; 0 failed; 0 skipped; finished in 21.70ms (50.86ms CPU time)

Ran 6 tests for test/libraries/AttestationTrailer.t.sol:AttestationTrailerTest
[PASS] testFuzz_parseTotalAndFailClosed(bytes,uint256) (runs: 256, : 9346, ~: 9445)
[PASS] test_decode_revertsOnMalformedEnvelope() (gas: 8705)
[PASS] test_decode_revertsOnWrongPayloadSize() (gas: 11523)
[PASS] test_decode_roundTrips() (gas: 13338)
[PASS] test_hasTrailer_falseWhenAbsent() (gas: 2813)
[PASS] test_hasTrailer_trueForWellFormed() (gas: 5275)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 20.04ms (20.05ms CPU time)

Ran 18 test suites in 35.18ms (181.64ms CPU time): 304 tests passed, 0 failed, 0 skipped (304 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.